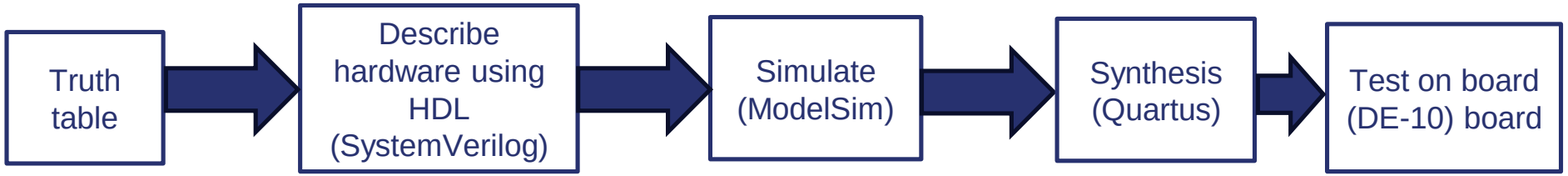
Lab 3: Introduction to hardware description language(cont)

Design flow

In this course: To design a digital circuit



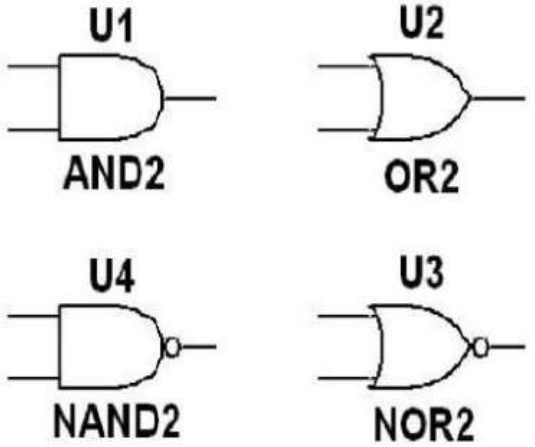
Continuous assignment

Continuous assignment: value of the left-hand side signal is continuously updated to match the value of the right-hand side expression
assign statement is used to create continuous assignment in hardware description.

Syntax:

assign LHS = RHS

LHS: signal need to assign a value to
RHS: signal or expression



Operator and precedence

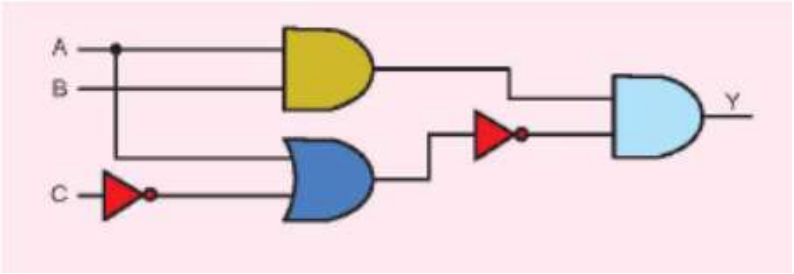
SystemVerilog operators and precedence of operators are shown in below table:

	Op	Meaning
Highest	~	NOT
	*, /, %	MUL, DIV, MOD
	+, -	PLUS, MINUS
	<<, >>	Logical Left/Right Shift
	<<<, >>>	Arithmetic Left/Right Shift
	<, <=, >, >=	Relative Comparison
	==, !=	Equality Comparison
	&, ~&	AND, NAND
Lowest	^, ~^	XOR, XNOR
	|, ~|	OR, NOR
	?:	Conditional

Exercises

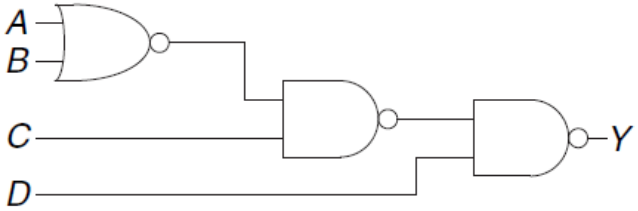
Ex1: Design a AND,NAND,OR,NOR gate using SystemVerilog, simulate on ModelSim and test operations of design in DE-10 board.

Ex2: Design the below circuit using SystemVerilog, simulate on ModelSim and test operations of design in DE-10 board.



Lối vào			Lối ra
A	B	C	Y
0	0	0	?
1	1	1	?

Ex3: Design the below circuit using SystemVerilog, simulate on ModelSim and test operations of design in DE-10 board.

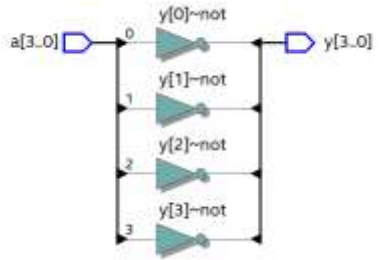


Bitwise Operators

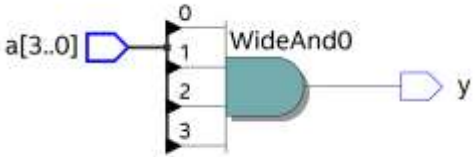
Bitwise operators act on single-bit signals or on multi-bit busses.

logic [3:0] a = ????

```
module lab1(input logic [3:0] a, output logic [3:0] y);
  assign y = ~a;
endmodule
```




```
module lab1(input logic [3:0] a, output logic y);
  assign y = &a;
// &a is much easier to write than
// assign y = a[3] & a[2] & a[1] & a[0];
endmodule
```



Operator and precedence

SystemVerilog operators and precedence of operators are shown in below table:

Numbers	Bits	Base	Val	Stored
3'b101	3	2	5	101
'b11	?	2	3	000 ... 0011
8'b11	8	2	3	00000011
8'b1010_1011	8	2	171	10101011
3'd6	3	10	6	110
6'o42	6	8	34	100010
8'hAB	8	16	171	10101011
42	?	10	42	00 ... 0101010

Conditional assignment

Conditional assignment: select the output from among alternatives based on an input called the condition
Using *assign* statement and conditional operator *?:*

Syntax:
assign y= <first exp> ?: <second exp> : <third exp>

Based on value of first exp (condition) to choose second exp or third exp

Example:

```
module mux_2to1(output y, input d0, d1, input sel);
                assign y = sel ? d1 : d0;
endmodule
```

Write testbench using Bitwise

lab1.sv | Compilation Report - lab1 | lab1_tb.sv

```
1  module lab1_tb();
2      // Khai báo tín hiệu cho testbench
3      logic [3:0] a;   // Đầu vào 4-bit
4      logic [3:0] y;   // Đầu ra 4-bit
5
6      // Khởi tạo module lab1
7      lab1 DUT(a,y);
8
9      // Khối initial để chạy các thử nghiệm
10     initial begin
11         a = 4'b0000;
12         #10;
13
14         a = 4'b1111;
15         #10;
16
17         a = 4'b1010;
18         #10;
19
20         a = 4'b0101;
21         #10;
22
23         a = 4'b0011;
24         #10;
25     end
26 endmodule
```

Pin assignments

STEP PINs	FPGA PINs	STEP PINs	FPGA PINs	Digital Display1	FPGA PINs	12M CLOCK	FPGA PINs
3.3V		VBUS		SEG-A1	E1	PCLK	J5
GPIO0	M4	GPIO35	B4	SEG-B1	D2	LED	FPGA PINs
GPIO1	P3	GPIO34	A5	SEG-C1	K2	LED1	N15
GPIO2	M5	GPIO33	A7	SEG-D1	J2	LED2	N14
GPIO3	R3	GPIO32	B6	SEG-E1	G2	LED3	M14
GPIO4	L6	GPIO31	E7	SEG-F1	F5	LED4	M12
GPIO5	P4	GPIO30	D7	SEG-G1	G5	LED5	L15
GPIO6	L7	GPIO29	B7	SEG-DP1	L1	LED6	K12
GPIO7	R5	GPIO28	C8	SEG-DIG1	E2	LED7	L11
GPIO8	P6	GPIO27	B8	Digital Display2	FPGA PINs	LED8	K11
GPIO9	R7	GPIO26	D10			Switch	FPGA PINs
GPIO10	P7	GPIO25	A9	SEG-A2	A3	SW1	J12
GPIO11	P8	GPIO24	A11	SEG-B2	A2	SW2	H11
GPIO12	P9	GPIO23	A13	SEG-C2	P2	SW3	H12
GPIO13	R9	GPIO22	B11	SEG-D2	P1	SW4	H13
GPIO14	R11	GPIO21	A14	SEG-E2	N1	Button	FPGA PINs
GPIO15	P12	GPIO20	B13	SEG-F2	C1	KEY1	J9
GPIO16	R14	GPIO19	B14	SEG-G2	C2	KEY2	K14
GPIO17	P15	GPIO18	B15	SEG-DP2	R2	KEY3	J11
GND		GND		SEG-DIG2	B1	KEY4	J14
RGB LED1	R	G	B	RGB_LED2	R	G	B
FPGA PINs	G15	E15	E14	FPGA PINs	C15	C14	D12

Exercises

Ex1: Design a 2:1 multiplexer using SystemVerilog, simulate on ModelSim and test operations of design in DE-10 board. (Using conditional operator)

Ex2: Design a 3:1 multiplexer using SystemVerilog, simulate on ModelSim and test operations of design in DE-10 board. (Using conditional operator)

Ex3: Design a 4:1 multiplexer using SystemVerilog, simulate on ModelSim and test operations of design in DE-10 board. (Using conditional operator)

Structural modeling

- Previous section discuss behavioral modeling
- Behavioral modeling: describing a module in terms of relationships between inputs and outputs
- Complex circuit -> Using structural modeling
- Strutural modeling: describing a module in terms of how it is combined from simpler module.

Ex: Design a 4: 1 multiplexer using 2:1 multiplexer

4:1 multiplexer:
4 data input: in1, in2, in3, in4
2 control input: ctr1, ctr2

```
module mux4(input logic in1,in2,in3,in4,
            input logic ctr1, ctr2,
            output logic out);
assign out = ctr1 ? (ctr2 ? in4 : in3) : (ctr2 ? in2 : in1);
endmodule
```

```
module   mux2(input logic in1, in2, ctr
                        output logic out);

         assign out = ctr ? in1 : in2;
endmodule
```

=> Write a 4: 1 multiplexer using 3 2:1 multiplexer.

Exercise 1:

Design a 4:1 multiplexer using 3 2:1 multiplexer in SystemVerilog, simulate on ModelSim and test operations of design in DE-10 board.